Here are the instructions for building your final project:

# 1- The App

This project is about building a machine-learning-powered app using DevOps and MLOps. So we first need an app. You can feel free to develop any app topic you find interesting, and use any machine-learning model you want, as long as you train the model or fine-tune it. Training/fine-tuning the model reauires that you have a dataset ready, and potentially multiple versions of the same dataset. Data versioning is discussed in section 3.

Your app needs to be properly tested as well, have at least 3 meaningful tests implemented for your application, each having multiple edge cases for the respective functionality being tested.

# 2- DevOps

You are required to apply DevOps in your app development process, from start to finish. DevOps must be integrated in your app implementation with your team members.

## Version Control

Your development process must include a version control tool (Git) and an online repository (Github). From the begining of your project, plan your Git branching scheme (main, staging, dev features...), with the proper merging path (from feature to dev to staging to main).

## Docker

Your final app must be built on Docker, with a proper slim `Dockerfile` which requires that you also configure your `.dockerignore` file to ensure non-relevant files are not included in the final Docker image.

## CI/CD

You must configure automated Continuous Integration (CI) and Continuous Delivery/Deployment (CD).

Requirements:

- You must use a pipeline runner (GitHub Actions).
- On every pull request to any protected branch (e.g. `dev`, `staging`, `main`), the CI pipeline must:
  - Run the full test suite.
  - Run linting/formatting/static analysis (e.g. black/ruff/flake8/pylint...).
  - Build the Docker image and ensure it succeeds (no runtime build errors).
- A PR cannot be merged if CI fails.

Deployment requirements:

- You must have at least one automated deployment pipeline stage.
- Recommended flow:

    - Merges to `staging` → uses GitHub secrets specific for the staging environment to create a staging `.env` file, then build the staging Docker image. No release process is requested.
    - Merges to `main` → uses GitHub secrets specific for the production environment to create a production `.env` file, then build the production Docker image. No deployment process is requested.

Artifacts:

- You must include in the repo:
    - CI config (`.github/workflows/...yml`).
    - Documentation in `README.md` describing:
        - How code goes from commit → build → test → staging → prod.
        - Which branches trigger which pipelines.

---

# 3- MLOps

This section covers the machine learning lifecycle: data, experiments, models, and deployment of models. You are required to integrate basic MLOps concepts into your project using DVC and MLflow.

## Data Versioning (DVC)

You are required to track your training data with DVC (Data Version Control).

Goals:

- Treat data like code: version it, branch it, reproduce it.
- Be able to answer: "which exact dataset version produced this model?"

Requirements:

1. Initialize DVC in your repo to track the data used in your model training/fine-tuning.
2. Configure remote storage for DVC (MS Azure). This is where large data actually lives. Git should only store small `.dvc` pointer files, not the raw data.
3. You must maintain at least 2 dataset versions:
    - **v1:** first usable dataset you trained on.
    - **v2:** some improved/cleaned/augmented/split-tweaked version. It's OK if they're small (toy scale). The point is the process, not the dataset size.
4. You must document in `ml/README_data.md`:
    - Where the data comes from (source).
    - What changed from v1 → v2 (e.g. removed outliers, added synthetic samples, balanced classes, etc.).
    - Which commit hash corresponds to which dataset version (link code → data).

**Note:** Do NOT commit raw private data to GitHub. Use `.gitignore` and `.dvc` properly.

## Experiment Tracking (MLflow)

You must track your ML experiments using MLflow.

What to log per run:

- Code version: git commit SHA (can be logged manually as a param or tag).
- Dataset version: DVC dataset version ID or hash.
- Hyperparameters: learning rate, batch size, model architecture choices, etc.
- Metrics: accuracy, F1, RMSE, BLEU, whatever is relevant to your task.
- Artifacts: trained model weights, confusion matrices, plots (the pkl file or any relevant model save).

Expected workflow:

1. You run training code locally or in a container.
2. The training script automatically calls MLflow APIs to `mlflow.log_param`, `mlflow.log_metric`, `mlflow.log_artifact`.
3. MLflow produces a run history (local file backend is fine).

Deliverables for grading:

- Training script with MLflow logging calls.
- At least 2 distinct experiments tracked in MLflow:
  - Baseline model.
  - Improved / tuned / fine-tuned model.

You must include a short summary in `ml/README_experiments.md` explaining:

- Which run you consider production-worthy and why.
- The metric you're optimizing for and why that metric matters for your app.

---

# 4- Repo Structure (expected high-level layout)

Example structure you should aim for (names can vary, but responsibilities must be clear):

```
.
├── app/
│   ├── main.py                 # service entrypoint / API server
│   ├── your app files here ...
│   └── ...
├── ml/
│   ├── train.py                # training script (calls MLflow)
│   ├── data_pipeline.py        # preprocessing / feature engineering
│   ├── configs/                # hyperparams, training configs
│   ├── registry/
│   │   └── current_model.md   # which model is prod
│   ├── README_data.md
│   ├── README_experiments.md
│   ├── PROVENANCE.md
│   └── ...
├── data.dvc / data/            # data pointers (not raw data in git)
├── requirements.txt / pyproject.toml
├── Dockerfile                  # runtime/inference image (prod)
```

```
├── .dockerignore
├── .github/workflows/ci.yml
├── DEPLOYMENT.md               # how staging/prod deploy works
└── README.md                   # high-level overview
```